YNOV · MASTER 2 · CONTENEURISATION DOCKER

RAPPORT TECHNIQUE

# Conteneurisation d'une application Web multi-tiers

PHP 8.2 / Apache · MySQL 8.4 · Docker Compose

**Jérémy CHAN FUNG TING**
**Yann DUCLOS**

github.com/Mieru-yo/docker-php-articles
Octobre 2026

# Sommaire


Introduction 3

1 Architecture 3

2 Images Docker 4

2.1 Dockerfile du service Web 4

2.2 Dockerfile de la base de données 6

3 Orchestration Docker Compose 7

4 Réseaux et persistance 9

5 Sécurité de l'image Web 10

6 Optimisation de l'image Web 12

7 CI/CD et Docker Hub 15

7.1 Pipeline CI/CD (GitHub Actions) 15

7.2 Publication manuelle sur Docker Hub 17

8 Déploiement depuis un clone neuf 18

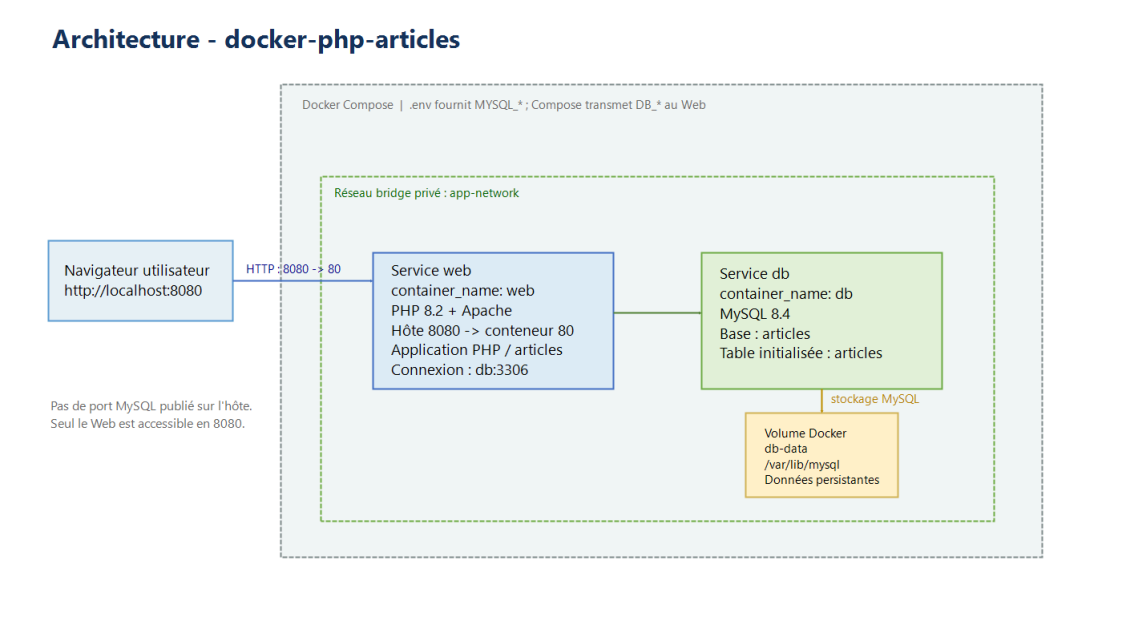
# Introduction

Conteneurisation d'une application Web PHP / MySQL (liste d'articles) : une image par service, orchestration avec Docker Compose, sécurisation et optimisation des images, pipeline CI/CD et publication sur Docker Hub.

| | |
|---|---|
| Dépôt Git | https://github.com/Mieru-yo/docker-php-articles |
| Image Web | https://hub.docker.com/r/nellotsuka/docker-php-articles-web |
| Image base de données | https://hub.docker.com/r/nellotsuka/docker-php-articles-db |

# 1. Architecture

## Vue d'ensemble




Source éditable du schéma : `docs/architecture.drawio`.

La stack est composée de deux services isolés sur le réseau bridge `app-network` :

- `web` (`container_name: web`) exécute l'application PHP 8.2 sous Apache. Le port `8080` de l'hôte est publié vers le port `80` du conteneur.
- `db` (`container_name: db`) exécute MySQL 8.4. La base `articles` est initialisée avec la table `articles`. Aucun port MySQL n'est publié sur l'hôte.

L'application rejoint la base sur le réseau interne à l'adresse `db:3306`. Le volume nommé `db-data` conserve les données MySQL dans `/var/lib/mysql` lorsque les conteneurs sont remplacés.

## Configuration

Les paramètres MySQL sont conservés dans le fichier local `.env`, créé à partir de `.env.example` et exclu du contrôle de version. Compose configure le serveur avec les variables `MYSQL_*` et transmet au service Web leurs équivalents applicatifs `DB_*`. Le code PHP ne contient donc aucun identifiant de connexion fixe.

# 2. Images Docker

## 2.1 Dockerfile du service Web

Fichier : `docker/web/Dockerfile`, construit depuis la racine du dépôt :

```
docker build -t docker-php-articles-web -f docker/web/Dockerfile .
```

### Corrections préalables du code PHP

Avant de conteneuriser l'application, trois problèmes du code fourni ont été corrigés :

| Problème | Correction |
| --- | --- |
| Hôte `localhost` et identifiants `test/test` codés en dur dans `db-config.php` : impossible de joindre le conteneur `db`. | Lecture de `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` depuis l'environnement. L'application refuse de démarrer si une variable manque. |
| `validation.php` testait `$_POST["title"]` au lieu de `$_POST["author"]`. | Condition corrigée. |
| Titre, contenu et auteur affichés sans échappement : un article contenant `<script>` était exécuté (faille XSS). Les erreurs PHP et le message PDO (hôte, utilisateur) étaient affichés au visiteur. | `htmlspecialchars` sur chaque valeur affichée, `display_errors` désactivé, message d'erreur neutre (le détail part dans les logs du conteneur). |

### Justification des instructions

| Instruction | Justification |
|---|---|
| `FROM php:8.2-apache AS build` | Image officielle PHP 8.2 avec Apache intégré, demandée par le sujet. Apache et PHP dans le même conteneur : un seul service Web à gérer. Sert d'étape de préparation (multi-stage, voir partie 6). |
| `RUN docker-php-ext-install pdo_mysql mysqli && apt-get purge … && rm -rf …` | Extensions nécessaires pour se connecter à MySQL ( `pdo_mysql` est utilisée par l'application, `mysqli` est demandée par le sujet). Dans la même couche, les outils de compilation fournis par l'image ( `gcc` , `make` , en-têtes du noyau…) sont supprimés, ainsi que les listes `apt` et les sources PHP : ils ne servent plus et ajoutaient 61 vulnérabilités (voir partie 5). |
| `COPY … zz-security-hardening.conf` + `a2enconf` | Configuration Apache : `ServerTokens Prod` et `ServerSignature Off` masquent la version d'Apache, `ServerName` supprime l'avertissement au démarrage. Le préfixe `zz-` la fait charger après `security.conf` de Debian, qui sinon remettait la version complète. |
| `COPY … zz-security.ini` | `expose_php = Off` : supprime l'en-tête `X-Powered-By` qui révélait la version de PHP. |
| `mv php.ini-production php.ini` | Configuration PHP de production (erreurs non affichées, limites adaptées). |
| `COPY --chown=root:root --chmod=644 app/ /var/www/html/` | Copie le code dans le DocumentRoot par défaut d'Apache. Les fichiers appartiennent à `root` et sont en lecture seule pour `www-data` : même si l'application était compromise, le processus Web ne pourrait pas modifier le code. |
| `FROM scratch` + `COPY --from=build / /` | Étape finale : le système de fichiers préparé est copié dans une image vide. Les fichiers supprimés pendant la préparation ne restent dans aucune couche : l'image passe de 176 à 63 Mo compressés (partie 6). |
| `ENV PATH=… PHP_INI_DIR=… APACHE_CONFDIR=… APACHE_ENVVARS=…`, `WORKDIR /var/www/html` | La copie ne reprend pas la configuration de l'image : on redéclare celle de `php:8.2-apache` dont PHP et Apache ont besoin. |
| `USER www-data` | Apache tourne en utilisateur non-root. Une faille dans l'application ne donne pas les droits root dans le conteneur. |
| `EXPOSE 80` | Documente le port d'écoute d'Apache. Pour qu'un utilisateur non-root puisse écouter sur le port 80, le Compose ajoute le `sysctl net.ipv4.ip_unprivileged_port_start=0` (voir partie 3). |
| `STOPSIGNAL SIGWINCH` | Signal d'arrêt propre d'Apache, repris de l'image officielle : Apache termine les requêtes en cours avant de s'arrêter. |
| `HEALTHCHECK … curl -fsS http://localhost/` | Docker vérifie toutes les 30 s que la page répond. La page répond en 200 même si la base est indisponible : l'état du conteneur Web reste indépendant de celui de la base. |
| `ENTRYPOINT ["docker-php-entrypoint"]`, `CMD ["apache2-foreground"]` | Point d'entrée et commande de l'image officielle : Apache est lancé au premier plan, comme processus principal du conteneur. |

### Pourquoi pas de `chmod 777`

`chmod 777` donne à **tous** les utilisateurs le droit de lire, modifier et exécuter les fichiers. Dans un conteneur Web, un attaquant qui exploite une faille de l'application pourrait alors remplacer le code PHP (par exemple déposer une porte dérobée). Ici, le code appartient à `root` en `644` : `www-data` peut seulement le lire.

**`.dockerignore`**

Le contexte de build est la racine du dépôt, partagée avec l'image de la base. Le `.dockerignore` fonctionne en liste blanche : tout est exclu (`*`), sauf `app/`, `db/init/` et les deux fichiers de configuration du service Web. Le `.git`, la documentation et surtout le fichier `.env` (mots de passe) ne sont jamais envoyés au daemon Docker.

### Tests

Image lancée avec le durcissement prévu pour le Compose (`--read-only`, `tmpfs`, `--cap-drop ALL`, `sysctl`, `no-new-privileges`) et une base MySQL 8.4 temporaire :

```
$ docker exec j2test-web id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' -X POST http://127.0.0.1:8090/validation.php -d
'title=Test J2&author=Jeremy&content=Image Web durcie'
HTTP 302

$ docker exec j2test-web sh -c 'touch /var/www/html/pirate.php'
touch: cannot touch '/var/www/html/pirate.php': Read-only file system

$ docker inspect j2test-web --format '{{.State.Health.Status}}'
healthy

$ curl -sI http://127.0.0.1:8090/ | grep -i -E "^(HTTP|server|x-powered-by)"
HTTP/1.1 200 OK
Server: Apache
```

## 2.2 Dockerfile de la base de données

### Dockerfile

Le fichier docker/db/Dockerfile part de l'image officielle `mysql:8.4`. Il copie le script d'initialisation sous `/docker-entrypoint-initdb.d/10-articles.sql`, emplacement traité par l'entrypoint officiel lors de l'initialisation d'un répertoire de données vide.

Le script SQL ne crée que la table `articles`. La base et l'utilisateur applicatif sont créés par l'image officielle à partir des variables `MYSQL_DATABASE`, `MYSQL_USER` et `MYSQL_PASSWORD` fournies par Compose. La table utilise InnoDB et l'encodage `utf8mb4`.

Le fichier appartient à `root` et est lisible sans être modifiable (`0644`). Aucun mot de passe n'est incorporé à l'image. Le contexte de build reste la racine du dépôt afin de pouvoir copier `db/init/articles.sql`.

### Vérification

Image construite et démarrée avec `docker compose up -d --build db` (fichier `.env` copié depuis `.env.example`) :

```
$ docker compose ps
NAME   IMAGE                                  SERVICE   STATUS
db     nellotsuka/docker-php-articles-db:latest   db        Up 45 seconds (healthy)   3306/tcp,
33060/tcp

$ echo "SHOW TABLES;" | docker exec -i db sh -c 'mysql -uapp -p"$MYSQL_PASSWORD" articles'
Tables_in_articles
articles
```

La structure obtenue confirme InnoDB et `utf8mb4` :

```
CREATE TABLE `articles` (
  `id` int unsigned NOT NULL AUTO_INCREMENT,
  `title` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `author` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `date` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `content` text COLLATE utf8mb4_unicode_ci NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

La base `articles` et l'utilisateur `app` sont donc bien créés par l'image MySQL, et le script ne crée que la table.

# 3. Orchestration Docker Compose

Le docker-compose.yml définit deux services (`web` et `db`), le réseau bridge `app-network` et le volume nommé `db-data`.

## Configuration

Les quatre variables `MYSQL_*` sont définies dans le fichier local `.env`, à créer à partir de .env.example. Les mots de passe de l'exemple sont des valeurs fictives à remplacer ; `.env` est ignoré par Git. Une variable manquante fait échouer `docker compose` (syntaxe `${VAR:?message}`).

## Service `db`

| Élément | Valeur | Justification |
|---|---|---|
| Image | construite par `docker/db/Dockerfile` | MySQL 8.4 avec le script de création de la table |
| `container_name` | `db` | nom fixe, utilisé par `docker exec db …` |
| Volume | `db-data` → `/var/lib/mysql` | les données survivent à la suppression du conteneur |
| Ports | aucun publié | la base n'est joignable que depuis `app-network` |
| Healthcheck | `mysqladmin ping -h 127.0.0.1` | permet à `web` d'attendre que MySQL soit prêt |

## Service `web`

| Élément | Valeur | Justification |
|---|---|---|
| Image | construite par `docker/web/Dockerfile` | PHP 8.2 + Apache, non-root |
| Port | `8080:80` | application sur `http://localhost:8080` |
| Variables | `DB_HOST=db`, `DB_PORT=3306`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` | seules variables lues par le code PHP ; les trois dernières viennent des `MYSQL_*` |
| `depends_on` | `db` avec `condition: service_healthy` | le Web démarre après que la base répond |
| `read_only: true` + `tmpfs` | `/tmp`, `/var/run/apache2`, `/var/lock/apache2` | système de fichiers en lecture seule, seuls les dossiers temporaires d'Apache sont inscriptibles |
| `cap_drop: ALL` | toutes les capabilities retirées | réduit la surface d'attaque |
| `sysctls` | `net.ipv4.ip_unprivileged_port_start: 0` | permet à `www-data` d'écouter sur le port 80 sans capability |
| `no-new-privileges` | activé | interdit toute élévation de privilèges |

Le healthcheck du service Web est défini dans son Dockerfile.

## Vérification

La stack complète a été démarrée avec `docker compose up -d --build` :

```
$ docker compose ps
NAME   IMAGE                                          SERVICE   STATUS                          PORTS
db     nellotsuka/docker-php-articles-db:latest    db        Up About a minute (healthy)
3306/tcp, 33060/tcp
web    nellotsuka/docker-php-articles-web:latest   web       Up About a minute (healthy)
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Test de l'application (création d'un article, puis lecture de la page d'accueil) :

```
$ curl -s -o NUL -w "GET / -> HTTP %{http_code}" http://localhost:8080/
GET / -> HTTP 200
$ curl -s -o NUL -w "POST -> HTTP %{http_code}" -X POST http://localhost:8080/validation.php -d
"title=Test Y3&author=Yann&content=Stack complete"
POST -> HTTP 302
$ curl -s http://localhost:8080/ | grep "Test Y3"
            <h2 class="h3">Test Y3 <small class="text-muted font-italic"></h2>
```

Vérification du durcissement :

```
$ docker exec web id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ docker exec web sh -c "touch /var/www/html/x"
touch: cannot touch '/var/www/html/x': Read-only file system
```

Le port MySQL n'étant pas publié, seul le service Web est accessible depuis l'hôte. Le test de persistance du volume est présenté dans la partie 4.

# 4. Réseaux et persistance

## Persistance des données

Le volume nommé `db-data` est monté sur `/var/lib/mysql`. Pour le vérifier, un article est créé, les conteneurs sont supprimés (`docker compose down`, sans l'option `-v`), puis recréés :

```
$ curl.exe -s -o NUL -w "POST -> HTTP %{http_code}" -X POST http://localhost:8080/validation.php
-d "title=Persistance&author=Yann&content=Doit survivre a la suppression des conteneurs"
POST -> HTTP 302

$ curl.exe -s http://localhost:8080/ | Select-String "Persistance"
            <h2 class="h3">Persistance <small class="text-muted font-italic"></h2>

$ docker compose down

$ docker volume ls --filter name=db-data
DRIVER    VOLUME NAME
local     db-data

$ docker compose up -d

$ curl.exe -s http://localhost:8080/ | Select-String "Persistance"
            <h2 class="h3">Persistance <small class="text-muted font-italic"></h2>
```

Les conteneurs ont été détruits et recréés, mais le volume `db-data` est resté en place : l'article est toujours affiché. Les données sont donc indépendantes du cycle de vie des conteneurs. Seule la commande `docker compose down -v` supprime le volume.

## Réseau

Les deux services sont rattachés au réseau bridge `app-network`, défini dans le Compose :

```
$ docker network ls --filter name=app-network
NETWORK ID     NAME          DRIVER    SCOPE
d47a33318254   app-network   bridge    local

$ docker network inspect app-network --format "{{.Name}} driver={{.Driver}} containers: {{range
.Containers}}{{.Name}}={{.IPv4Address}} {{end}}"
app-network driver=bridge containers: db=172.18.0.2/16 web=172.18.0.3/16
```

### Communication entre `web` et `db`

Sur un réseau bridge défini par l'utilisateur, Docker fournit une résolution DNS par nom de service. Le conteneur `web` joint donc la base avec `DB_HOST=db`, sans adresse IP en dur :

```
$ docker exec web getent hosts db
172.18.0.2      db

$ docker exec -i web php   # script : fsockopen("db", 3306)
TCP db:3306 OK
db -> 172.18.0.2
```

Test avec `curl` depuis le conteneur `web` :

```
$ docker exec web curl -sS --max-time 5 http://db:3306
curl: (1) Received HTTP/0.9 when not allowed

$ docker exec web curl -sS --max-time 5 --http0.9 -o /dev/null -w "connexion vers %{remote_ip}:%
{remote_port} etablie\n" http://db:3306
connexion vers 172.26.0.2:3306 etablie

$ docker exec web curl -sS --max-time 5 --http0.9 http://db:3306 | cat -v
J^@^@^@
8.4.13^@ ... caching_sha2_password^@ ... Got packets out of order

$ docker exec web curl -sS --max-time 5 http://db:3307
curl: (7) Failed to connect to db port 3307 after 1 ms: Could not connect to server
```

MySQL ne parle pas HTTP : `curl` refuse la réponse, mais la connexion TCP vers `db:3306` est bien établie. Avec `--http0.9`, on voit le message d'accueil du serveur MySQL (version `8.4.13`, méthode d'authentification `caching_sha2_password`). À titre de comparaison, un port où rien n'écoute (`3307`) donne une erreur de connexion.

#### À propos de `ping`

`ping` n'est pas disponible dans le conteneur `web` :

```
$ docker exec web ping -c 2 db
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable
file not found in $PATH
```

L'image officielle `php:8.2-apache` ne contient pas `ping`. L'ajouter demanderait d'installer un paquet (`iputils-ping`), ce qui alourdit l'image, et `ping` utilise des sockets ICMP que le durcissement (`cap_drop: ALL`) restreint. Plutôt que d'affaiblir l'image, la connectivité est prouvée par la résolution DNS (`getent`) et par une vraie connexion TCP vers `db:3306`, ce qui est plus pertinent : c'est exactement ce que fait l'application.

#### Isolation

Le port MySQL n'est pas publié sur l'hôte, seul `8080` l'est (pour `web`) :

```
$ docker compose ps db --format "{{.Ports}}"
3306/tcp, 33060/tcp

$ Test-NetConnection localhost -Port 3306   # TcpTestSucceeded
False
```

La base n'est donc joignable que depuis `app-network`.

# 5. Sécurité de l'image Web

## Scan de vulnérabilités avec Trivy

L'image Web est scannée avec Trivy, lancé depuis son image officielle (aucune installation sur la machine) :

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest \
  image --scanners vuln --severity CRITICAL,HIGH docker-php-articles-web:latest
```

### Avant correction

```
docker-php-articles-web:avant-j3 (debian 13.7)
==============================================
Total: 120 (HIGH: 117, CRITICAL: 3)
```

| Paquet | Vulnérabilités | Correctif disponible |
|---|---|---|
| `linux-libc-dev` (en-têtes du noyau) | 61, dont 2 CRITICAL | non |
| `libxml2` | 8, dont 1 CRITICAL | non |
| autres (`bsdutils`, `libblkid1`, `libmount1` …) | 51 | non |

Aucune des 120 vulnérabilités n'a de version corrigée publiée dans Debian 13 : un `apt-get upgrade` ne change rien. La correction consiste donc à **supprimer les paquets inutiles** à l'exécution.

### Correction

L'image officielle `php:8.2-apache` conserve tous les outils de compilation (`$PHPIZE_DEPS` : `gcc`, `g++`, `make`, `autoconf` …, plus `libc6-dev` et `linux-libc-dev`) pour pouvoir compiler des extensions. Une fois `pdo_mysql` et `mysqli` compilées, ils ne servent plus et augmentent la surface d'attaque. Ils sont supprimés dans la même instruction `RUN` que la compilation :

```
RUN docker-php-ext-install pdo_mysql mysqli \
    && apt-get purge -y --auto-remove \
        autoconf dpkg-dev file g++ gcc make pkg-config re2c libc6-dev \
    && rm -rf /var/lib/apt/lists/* /tmp/* /usr/src/php.tar.xz
```

En plus de réduire les vulnérabilités, un attaquant qui prendrait le contrôle du conteneur ne disposerait plus d'un compilateur.

### Après correction

```
docker-php-articles-web:apres-j3 (debian 13.7)
==============================================
Total: 59 (HIGH: 58, CRITICAL: 1)
```

| | CRITICAL | HIGH | Total |
|---|---|---|---|
| Avant | 3 | 117 | 120 |
| Après | 1 | 58 | 59 |

La vulnérabilité critique restante (`CVE-2026-6653`, `libxml2`) n'a pas encore de correctif publié par Debian, et `libxml2` est nécessaire à PHP : elle ne peut pas être supprimée. Elle disparaîtra en reconstruisant l'image quand Debian publiera le correctif (le pipeline CI reconstruit l'image à chaque push).

Rapports complets : `docs/rapport/sorties/5-trivy-avant-j3.txt` et `5-trivy-apres-j3.txt`.

### Mesures de sécurité de l'image Web

| Mesure | Où | Effet |
|---|---|---|
| Utilisateur non-root `www-data` | `USER www-data` (Dockerfile) | Une faille dans l'application ne donne pas les droits root |
| Code en lecture seule, propriétaire `root`, droits `644` | `COPY --chown=root:root --chmod=644` | Le processus Web ne peut pas modifier le code ; pas de `chmod 777` |
| Outils de compilation supprimés | `apt-get purge` (Dockerfile) | 61 vulnérabilités en moins, pas de compilateur pour un attaquant |
| Versions d'Apache et PHP masquées | `ServerTokens Prod`, `expose_php = Off` | Un attaquant ne peut pas cibler une version précise |
| Erreurs PHP non affichées, sorties échappées | code PHP, `php.ini-production` | Pas de fuite d'informations, pas de faille XSS |
| Aucun secret dans l'image | variables `DB_*` fournies à l'exécution | Les mots de passe ne sont ni dans le Dockerfile ni dans le code |
| Système de fichiers en lecture seule, `cap_drop: ALL`, `no-new-privileges` | `docker-compose.yml` (partie 3) | Limite ce qu'un conteneur compromis peut faire |

### Vérifications

Image lancée avec le durcissement prévu pour le Compose et la base MySQL du projet :

```
$ docker exec j3test-web id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

$ docker run --rm docker-php-articles-web:apres-j3 sh -c 'php -m | grep -E
"^(pdo_mysql|mysqli)$"; which gcc make || echo "gcc/make absents"'
mysqli
pdo_mysql
gcc/make absents

$ curl -s -o /dev/null -w 'ajout: HTTP %{http_code}\n' -X POST
http://127.0.0.1:8092/validation.php -d 'title=Test J3&author=Jeremy&content=Image sans outils de
compilation'
ajout: HTTP 302

$ docker inspect j3test-web --format '{{.State.Health.Status}}'
healthy
```

## 6. Optimisation de l'image Web

### Constat de départ

Après la partie 5, l'image Web ne contient plus d'outils de compilation, mais sa taille n'a pas baissé (mesures `docker images` et `docker image inspect`) :

| Image | Compressée | Sur disque | Couches |
|---|---|---|---|
| avant la partie 5 | 175,9 Mo | 708 Mo | 20 |
| après la partie 5 | 176,1 Mo | 709 Mo | 20 |

Une image Docker est un empilement de couches en lecture seule. Supprimer un fichier dans une couche ne fait que le masquer : il reste stocké dans la couche de `php:8.2-apache` où il a été créé. Les outils de compilation supprimés en partie 5 sont donc toujours téléchargés.

## Choix de l'image de base

| Option | Retenue | Raison |
|---|---|---|
| `php:8.2-apache` (Debian) | ✅ | Image officielle demandée par le sujet, Apache et PHP dans un seul processus. |
| `php:8.2-fpm-alpine` | ❌ | Image plus petite, mais PHP-FPM a besoin d'un serveur Web séparé (Nginx) : un deuxième service, ou deux processus dans le même conteneur. Changement d'architecture non justifié pour cette application. |

L'optimisation porte donc sur le contenu de l'image, pas sur la base.

## Multi-stage build

Le Dockerfile est découpé en deux étapes :

1. **Étape `build`** (`FROM php:8.2-apache`) : compilation des extensions, suppression des outils de compilation, des en-têtes PHP, de la documentation et des pages de manuel, configuration d'Apache et de PHP, copie du code.
2. **Étape finale** (`FROM scratch`) : image vide dans laquelle on copie le système de fichiers final de l'étape 1 (`COPY --from=build / /`). Les fichiers supprimés n'existent plus dans aucune couche.

`COPY --from=build / /` ne reprend pas la configuration de l'image (variables, point d'entrée…). Elle est redéclarée à partir de celle de `php:8.2-apache` : `PATH`, `PHP_INI_DIR`, `APACHE_CONFDIR`, `APACHE_ENVVARS`, `WORKDIR`, `STOPSIGNAL SIGWINCH`, `ENTRYPOINT ["docker-php-entrypoint"]`, `CMD ["apache2-foreground"]`.

Nettoyage effectué dans l'étape 1 :

| Supprimé | Pourquoi |
|---|---|
| `gcc`, `g++`, `make`, `autoconf`, `libc6-dev`, `linux-libc-dev` … | Outils de compilation, inutiles une fois les extensions compilées |
| `/usr/local/include/php`, `/usr/local/lib/php/build` | En-têtes et scripts de compilation d'extensions PHP |
| `/usr/src/php.tar.xz` | Archive des sources de PHP |
| `/var/lib/apt/lists/*`, `/var/cache/apt/*` | Listes et cache des paquets `apt` |
| `/usr/share/doc/*`, `/usr/share/man/*`, `/usr/share/info/*` | Documentation |

## Résultat

Tailles mesurées avec `docker image inspect --format '{{.Size}}'` (compressée), `docker images` (sur disque) et `docker image inspect --format '{{len .RootFS.Layers}}'` (couches) :

| | Taille compressée (téléchargée depuis Docker Hub) | Taille sur disque | Couches |
|---|---|---|---|
| Avant | 176,1 Mo | 709 Mo | 20 |
| Après | 62,6 Mo | 276 Mo | 2 |
| Gain | **−64 %** | **−61 %** | |

Le contenu de l'image est identique à celui de la partie 5 : le scan Trivy donne le même résultat.

```
$ trivy image --scanners vuln --severity CRITICAL,HIGH docker-php-articles-web:apres-j4
docker-php-articles-web:apres-j4 (debian 13.7)
Total: 59 (HIGH: 58, CRITICAL: 1)
```

`.dockerignore`

Le contexte de build est la racine du dépôt. Le `.dockerignore` fonctionne en liste blanche :

```
*
!app/
!db/init/
!docker/web/apache-security.conf
!docker/web/php-security.ini
```

Seuls le code PHP, le script SQL et les deux fichiers de configuration du service Web sont envoyés au daemon. Le dossier `.git`, la documentation, les captures d'écran et le fichier `.env` (mots de passe) ne peuvent pas se retrouver dans une image par erreur.

## Vérification avec la stack complète

L'image optimisée a été testée avec le `docker-compose.yml` du projet :

```
$ docker compose ps
NAME      IMAGE                                      STATUS                    PORTS
db        nellotsuka/docker-php-articles-db:latest   Up 31 seconds (healthy)   3306/tcp,
33060/tcp
web       nellotsuka/docker-php-articles-web:latest  Up 8 seconds (healthy)    127.0.0.1:8095-
>80/tcp

$ curl -s -o /dev/null -w 'ajout: HTTP %{http_code}\n' -X POST
http://127.0.0.1:8095/validation.php -d 'title=Test J4&author=Jeremy&content=Image aplatie'
ajout: HTTP 302

$ docker exec web id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

$ docker exec web sh -c 'touch /var/www/html/x'
touch: cannot touch '/var/www/html/x': Read-only file system

$ docker stop web
web

$ docker inspect web --format '{{.State.ExitCode}}'
0
```

L'arrêt en 2 secondes avec le code 0 confirme que le signal d'arrêt `SIGWINCH` redéclaré est bien transmis à Apache.

# 7. CI/CD et Docker Hub

## 7.1 Pipeline CI/CD (GitHub Actions)

Fichier : `.github/workflows/docker.yml`.

### Déclenchement

Le pipeline se lance à chaque `push` (toutes les branches) et à chaque Pull Request vers `main`. Chaque modification est donc vérifiée avant et après sa fusion.

### Étapes

Le pipeline contient trois jobs, enchaînés avec `needs` : un job ne démarre que si le précédent a réussi.

| Job | Ce qu'il fait | Pourquoi |
|---|---|---|
| **Lint PHP** | `php -l` sur chaque fichier de `app/`, avec l'image `php:8.2-cli` | Détecte une erreur de syntaxe avant même de construire les images, avec la même version de PHP que l'image Web |
| **Build et test de la stack** | Copie `.env.example` en `.env`, lance `docker compose up -d --build --wait`, puis teste l'application | Construit les deux images et vérifie que la stack fonctionne réellement, pas seulement qu'elle se construit |
| **Publication Docker Hub** | Construit et pousse les images `web` et `db` avec les tags `latest` et le SHA court du commit | Ne s'exécute que sur un `push` vers `main` : seul le code fusionné est publié |

#### Tests exécutés par le job « Build et test »

1. `docker compose up --wait` attend que les **healthchecks** des deux conteneurs passent à `healthy` (échec au bout de 180 s).
2. La page d'accueil répond et contient le titre « Articles ».
3. Un article est ajouté via `validation.php` (réponse attendue : redirection `302`), puis on vérifie qu'il apparaît sur la page d'accueil : le Web lit et écrit bien dans la base.
4. Le processus du conteneur `web` ne tourne pas en root (`id -u` différent de `0`).
5. En cas d'échec, les logs des conteneurs sont affichés. La stack est toujours arrêtée à la fin (`docker compose down -v`).

### Secrets

La connexion à Docker Hub utilise deux secrets du dépôt GitHub (*Settings* → *Secrets and variables* → *Actions*) :

| Secret | Valeur |
|---|---|
| `DOCKERHUB_USERNAME` | `nellotsuka` |
| `DOCKERHUB_TOKEN` | jeton d'accès Docker Hub (*Read & Write*), pas le mot de passe du compte |

Aucun identifiant n'apparaît dans le fichier du pipeline. Si les secrets ne sont pas configurés, le job de publication affiche un avertissement et ne fait rien, au lieu d'échouer.

### Tags publiés

| Tag | Exemple | Usage |
|---|---|---|
| `latest` | `nellotsuka/docker-php-articles-web:latest` | Dernière version de `main`, utilisée par `docker-compose.yml` |
| SHA court du commit | `nellotsuka/docker-php-articles-web:21effd9` | Version figée : permet de revenir à une version précise |

### Exécution

Premier passage sur la branche `ci/github-actions` (run `40853016316`) :

```
## Lint PHP -> success
   php -l sur app/ -> success
## Build et test de la stack -> success
   Fichier .env de test -> success
   Build et démarrage (attend que les healthchecks soient OK) -> success
   État des conteneurs -> success
   La page d'accueil répond -> success
   Ajout d'un article et lecture depuis la base -> success
   Le conteneur web tourne en non-root -> success
   Logs en cas d'échec -> skipped
   Arrêt de la stack -> success
## Publication Docker Hub -> skipped
```

La publication est ignorée car il ne s'agit pas d'un push sur `main`.

#### Publication depuis `main`

Après l'ajout des deux secrets, le pipeline du commit `21effd9` sur `main` a été relancé (run `40880302492`, tentative 2) :

```
## Lint PHP -> success                  (13:46:06 -> 13:46:19)
## Build et test de la stack -> success  (13:46:22 -> 13:47:20)
## Publication Docker Hub -> success     (13:47:22 -> 13:48:28)
```

Tags relevés ensuite par l'API publique de Docker Hub (`https://hub.docker.com/v2/repositories/nellotsuka/<dépôt>/tags`) :

| Dépôt | Tag | Poussé le (UTC) | Taille compressée | Par | Digest |
|---|---|---|---|---|---|
| `docker-php-articles-web` | `latest` | 2026-10-29 13:48:02 | 62,6 Mo | `nellotsuka` (CI) | `sha256:10b4567fafd5…` |
| `docker-php-articles-web` | `21effd9` | 2026-10-29 13:48:03 | 62,6 Mo | `nellotsuka` (CI) | `sha256:10b4567fafd5…` |
| `docker-php-articles-db` | `latest` | 2026-10-29 13:48:16 | 238,3 Mo | `nellotsuka` (CI) | `sha256:6b84d9c4e0c6…` |
| `docker-php-articles-db` | `21effd9` | 2026-10-29 13:48:18 | 238,3 Mo | `nellotsuka` (CI) | `sha256:6b84d9c4e0c6…` |

`latest` et `21effd9` ont le même digest : ce sont deux noms pour la même image. Le tag `6c5b832`, poussé à la main par `yannd0cker` (partie 7.2), reste disponible dans les deux dépôts.

## 7.2 Publication manuelle sur Docker Hub

Les deux images sont publiées à la main sur Docker Hub, dans les dépôts publics du compte `nellotsuka`, avec le compte `yannd0cker` (collaborateur des dépôts) :

| Image | Dépôt Docker Hub |
|---|---|
| Web (PHP 8.2 + Apache) | `nellotsuka/docker-php-articles-web` |
| Base (MySQL 8.4) | `nellotsuka/docker-php-articles-db` |

### Procédure

Les images portent déjà leur nom complet grâce à la clé `image:` du `docker-compose.yml`. Elles sont construites par Compose, puis chacune reçoit un second tag : le SHA court du commit publié.

```
docker login
docker compose build
SHA=$(git rev-parse --short HEAD)
docker tag nellotsuka/docker-php-articles-web:latest nellotsuka/docker-php-articles-web:$SHA
docker tag nellotsuka/docker-php-articles-db:latest  nellotsuka/docker-php-articles-db:$SHA
docker push nellotsuka/docker-php-articles-web:latest
docker push nellotsuka/docker-php-articles-web:$SHA
docker push nellotsuka/docker-php-articles-db:latest
docker push nellotsuka/docker-php-articles-db:$SHA
```

Deux tags par image :

- `latest` : la dernière version, celle qu'utilise `docker compose pull`.
- le SHA court du commit (ici `6c5b832`) : une version précise et reproductible, qui permet de retrouver le code source exact.

`docker login` demande les identifiants dans le terminal ; aucun mot de passe ni jeton n'est écrit dans le dépôt.

### Vérification

Contenu des dépôts, relevé par l'API publique de Docker Hub (sans authentification) :

```
$ curl -s "https://hub.docker.com/v2/repositories/nellotsuka/docker-php-articles-web/tags"
latest   2026-10-29T13:05:03Z   59,7 MB   yannd0cker
6c5b832  2026-10-29T13:05:19Z   59,7 MB   yannd0cker

$ curl -s "https://hub.docker.com/v2/repositories/nellotsuka/docker-php-articles-db/tags"
latest   2026-10-29T13:07:46Z   227,2 MB  yannd0cker
6c5b832  2026-10-29T13:08:00Z   227,2 MB  yannd0cker
```

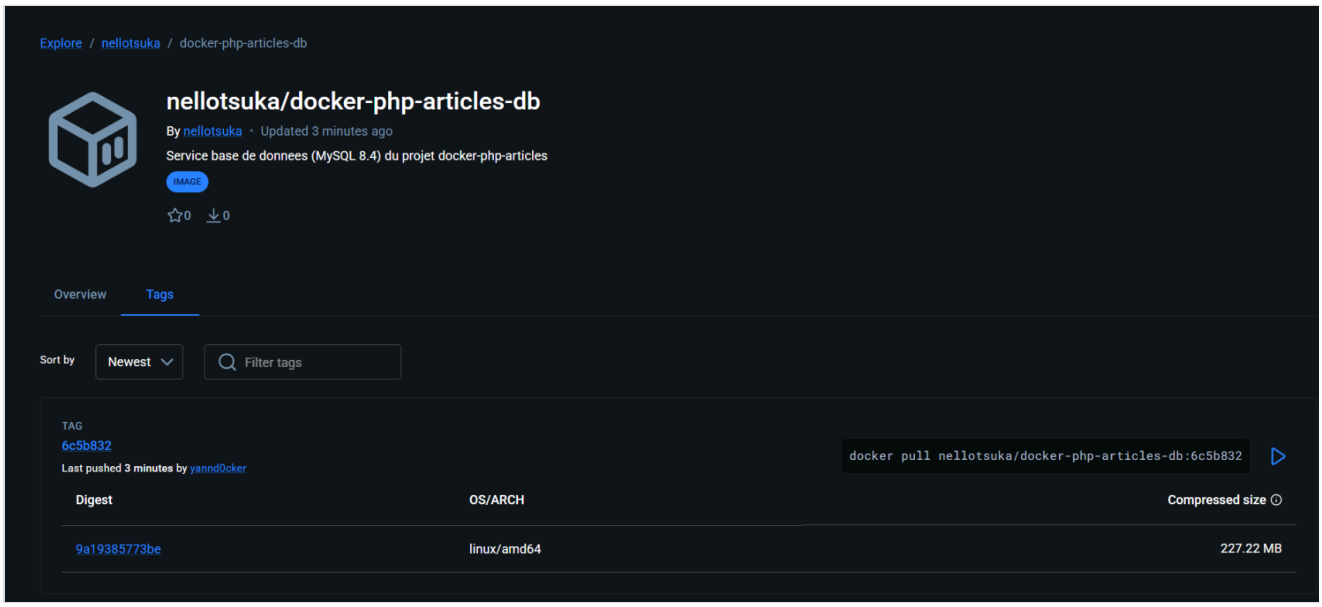
Les tailles sont celles des images compressées sur Docker Hub. En local, `docker images` affiche les tailles décompressées (276 Mo pour `web`, 1,1 Go pour `db`).

Le tag `latest` et le tag `6c5b832` ont la même taille : ils désignent la même image. Les dépôts sont publics, donc consultables et téléchargeables sans compte.

### Captures

Onglet *Tags* des deux dépôts publics, avec le tag `6c5b832` publié par `yannd0cker`, la plateforme `linux/amd64` et la taille compressée :




Les tailles compressées (59,73 Mo et 227,22 Mo) correspondent à celles relevées par l'API ci-dessus.

# 8. Déploiement depuis un clone neuf

Objectif : prouver que la stack démarre avec seulement Docker, Git et le dépôt, en utilisant les images publiées sur Docker Hub (aucun build local, aucun `docker login`).

## Environnement

```
> OS
Microsoft Windows 11 Famille 10.0.26200 (build 26200)
> docker version
Client: 29.0.1  Server: 29.0.1 (linux/amd64)
> docker compose version
Docker Compose version v2.40.3-desktop.1
```

Préparation de la machine avant le test :

- les images `nellotsuka/docker-php-articles-web:latest` et `-db:latest` présentes en local ont été supprimées (`docker rmi`), pour que le `pull` les télécharge réellement ;
- aucun conteneur `web` / `db`, volume `db-data` ou réseau `app-network` n'existait.

## Clonage et configuration

Dans un dossier vide :

```
PS ...\scratchpad\test-machine-propre> git clone https://github.com/Mieru-yo/docker-php-
articles.git
Cloning into 'docker-php-articles'...
PS ...\test-machine-propre\docker-php-articles> git log --oneline -1
96078d3 Merge pull request #13 from Mieru-yo/docs/ci-publish
PS ...\test-machine-propre\docker-php-articles> Copy-Item .env.example .env
```

Les deux mots de passe d'exemple du `.env` ont été remplacés par des valeurs aléatoires de 24 caractères (masquées ci-dessous) :

```
MYSQL_ROOT_PASSWORD=********
MYSQL_DATABASE=articles
MYSQL_USER=app
MYSQL_PASSWORD=********
```

## Démarrage depuis Docker Hub

```
> docker images nellotsuka/docker-php-articles-*
IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA

> docker compose pull
 web Pulling
 db Pulling
 e15d78f48822 Downloading [>                                                  ]    684kB/62.63MB
 ...
 db Pulled
 e15d78f48822 Download complete
 e15d78f48822 Pull complete
 web Pulled

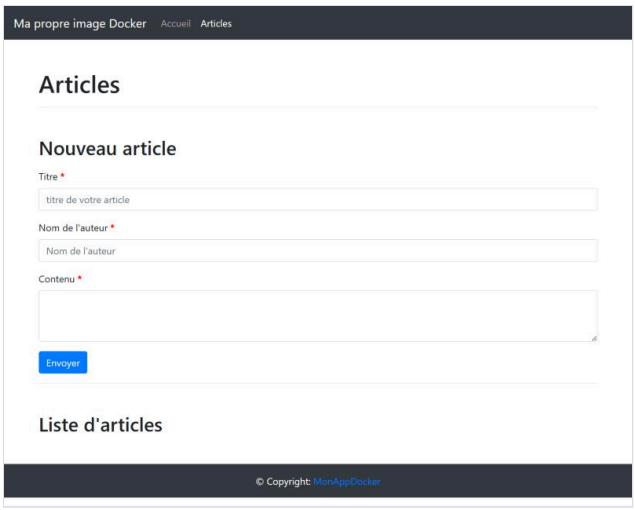
> docker compose up -d --no-build
 Network app-network  Created
 Volume db-data  Created
 Container db  Created
 Container web  Created
 Container db  Started
 Container db  Healthy
 Container web  Started

> docker compose ps
NAME   IMAGE                                    STATUS                   PORTS
db     nellotsuka/docker-php-articles-db:latest    Up 47 seconds (healthy)   3306/tcp, 33060/tcp
web    nellotsuka/docker-php-articles-web:latest   Up 25 seconds (healthy)   0.0.0.0:8080-
>80/tcp, [::]:8080->80/tcp
```

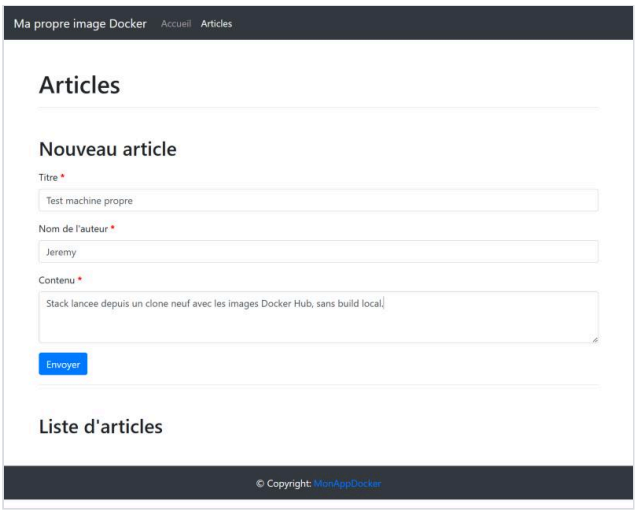
- L'image `web` (couche unique de 62,6 Mo, partie 6) est entièrement téléchargée.
- Pour `db`, seule la dernière couche (le script SQL) est téléchargée : les couches de `mysql:8.4` étaient déjà présentes sur la machine.
- `--no-build` garantit que Compose utilise les images de Docker Hub et ne reconstruit rien, même si le `docker-compose.yml` contient une section `build`.

## Utilisation de l'application

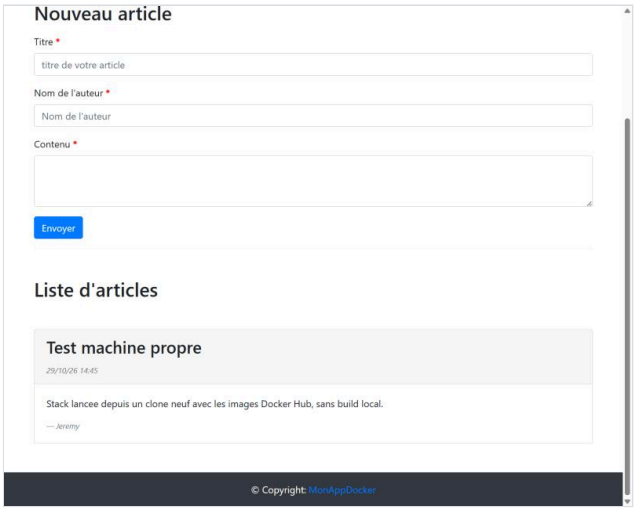
Captures du navigateur sur `http://localhost:8080/` :



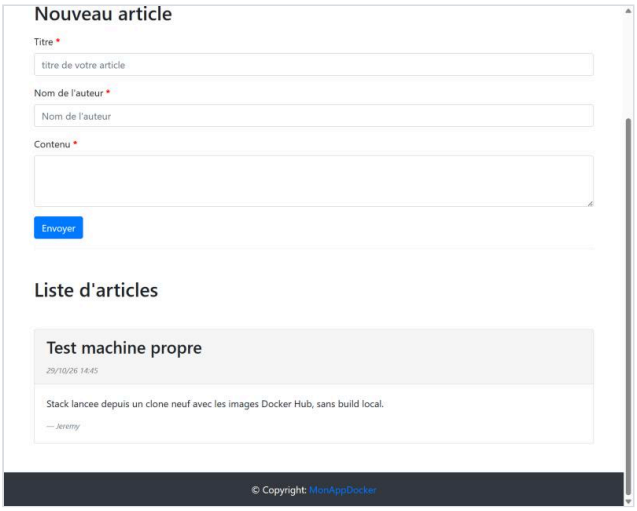

Page Articles, liste vide




Formulaire rempli




Article enregistré et affiché




Même page après `down` puis `up -d`

## Persistance

```
> docker compose down
 Container web  Removed
 Container db  Removed
 Network app-network  Removed

> docker volume ls --filter name=db-data
DRIVER    VOLUME NAME
local     db-data

> docker compose up -d --no-build
 Network app-network  Created
 Container db  Created
 Container web  Created
 Container db  Healthy
 Container web  Started

> docker compose ps
NAME   IMAGE                                        STATUS                    PORTS
db     nellotsuka/docker-php-articles-db:latest    Up 27 seconds (healthy)   3306/tcp, 33060/tcp
web    nellotsuka/docker-php-articles-web:latest   Up 21 seconds (healthy)   0.0.0.0:8080-
>80/tcp, [::]:8080->80/tcp

> curl.exe -s http://localhost:8080/ | Select-String 'Test machine propre'
<h2 class="h3">Test machine propre <small class="text-muted font-italic"></h2>
```

`docker compose down` supprime les conteneurs et le réseau, mais pas le volume `db-data` : l'article créé avant l'arrêt est toujours affiché après le redémarrage.